

LEGACY CODEBASE · AGENTIC SDLC RETROFIT · PRODUCTION-SAFE

5–8 Year-Old **Legacy** Code → Autonomous **SDLC** Pipeline

Scan existing microservices · Train Claude on your codebase · Generate stories from real code
Fix legacy code + tests · Add Playwright E2E · Deploy safely. Zero greenfield assumptions.

Phase 0: Prework

Code Archaeology

Train Claude

Legacy → Stories

Playwright E2E

The Fundamental Difference

Greenfield (Previous Guide)

- ✓ Start from Epic → generate everything
- ✓ Claude invents architecture fresh
- ✓ No existing code conflicts
- ✓ Clean test baseline (zero tests)
- ✓ No production risk

Legacy Retrofit (This Guide)

- **Start from CODE → derive stories**
- Claude learns YOUR architecture first
- Existing patterns must be respected
- Legacy tests may be broken/absent/wrong
- Every change risks breaking production
- **Customer integrations = extra risk surface**

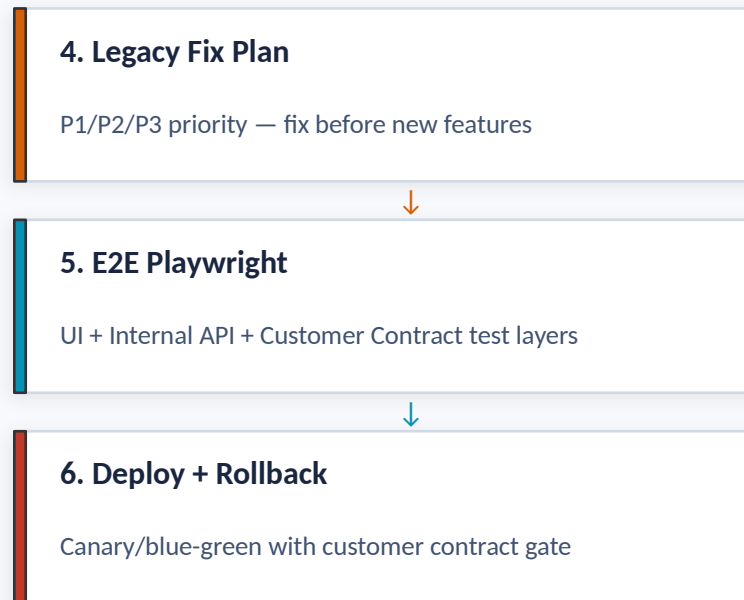
 **Core Rule:** Claude must READ before it WRITES. Prework creates the knowledge base so agents understand your codebase's patterns, antipatterns, and customer dependencies before generating a single story.

How Legacy SDLC Flow Differs from Greenfield

GREENFIELD:



LEGACY:



Prework Map — 6 Steps Before Any Agent Runs

01

Service Inventory

List all microservices, versions, owners, customer-facing status

02

Code Scan

Claude Code reads each service — tech stack, APIs, data models, integrations

03

Write CLAUDE.md Docs

Root + per-service context files. Claude's persistent knowledge base

04

Gap Analysis

What's missing: tests, validation, error handling, broken functionality

05

Risk Map

Dependency map, customer API contracts, danger zones, shared DB tables

06

Story Generation

Type A (reverse: existing behavior) + Type B (gap: what's missing/broken)

 Time budget: 2–3 days for 8–10 services. Not optional overhead — the difference between an agent that understands your system and one that breaks it.

Code Archaeology — What Claude Scans

Service Profile Scan

- ▶ Technology stack + dependency versions
- ▶ Every REST endpoint + consumer
- ▶ All @Entity classes + DB relationships
- ▶ Test count vs prod class count
- ▶ TODO/FIXME patterns + security red flags
- ▶ External API calls + event topics

Dependency Map

- ▶ Service → service call graph
- ▶ Shared DB tables (blast radius)
- ▶ Customer-facing API surface
- ▶ Message topic publish/consume map
- ▶ Critical paths (A→B→C→D chains)
- ▶ Danger zones: no circuit breaker

Test Health Scan

- ▶ JUnit 4 vs 5 per service
- ▶ Tests with zero assertions
- ▶ Currently FAILING tests (mvn test)
- ▶ @Ignored tests — why?
- ▶ Coverage % estimate per service
- ▶ TestContainers vs H2 usage

Training Claude on Your Codebase

💡 Key Concept: Claude has no persistent memory between sessions. "Training Claude" = creating well-structured files every agent reads at session start. Quality of files = quality of every story, fix, and test generated.

📄 CLAUDE.md (root)

- ▶ Critical rules: never modify customer APIs without approval
- ▶ Services map: name, port, Java version, DB, customer-facing?
- ▶ Existing patterns: exception handling, auth, DTO, testing style
- ▶ Customer API contracts: which endpoints are SLA-protected
- ▶ Shared library versions (changes cascade to ALL services)

📄 {service}/CLAUDE.md

- ▶ Service-specific quirks: known flags, legacy paths, odd configs
- ▶ Current known issues with JIRA ticket refs
- ▶ Test state: coverage %, ignored tests, hardcoded test data
- ▶ Safe zones: what can be changed with normal care
- ▶ Danger zones: what needs human approval before touching

📖 DOMAIN_GLOSSARY.md

- ▶ Exact business terms (User vs Customer vs Merchant)
- ▶ Business rules agents must know (30-day deactivation notice)
- ▶ What this system is NOT (not a payment processor)
- ▶ SLA constraints (customer API <200ms)
- ▶ PII fields and encryption requirements

How to Verify Claude Is Trained

Test with these 4 questions. All must be answered correctly without you explaining anything.

1

"What happens if I change `UserRepository.findByEmail()`?"

Expected: Lists all 4 downstream callers, flags customer risk, suggests safe approach

2

"Write a test for `UserService.createUser()` following our patterns"

Expected: JUnit 4 style, correct base class, H2, naming matches existing tests exactly

3

"Is it safe to add a nickname field to the User entity?"

Expected: Checks customer API contract, migration needed, shared library impact

4

"Create a story for adding order filtering by date range"

Expected: References existing filter patterns, flags no schema change needed, correct SP estimate

Story Generation FROM Code

Type A — Reverse Stories

Document what code CURRENTLY does.
Becomes source of truth for preserved behavior.
Used as regression protection baseline.

derived from code

no new dev needed

Type B — Gap Stories

What system SHOULD do but doesn't.
Missing tests, broken edge cases, absent features,
legacy code needing modernization.

requires dev work

sprint-planned

TYPE A — Reverse Stories

US-E001	[Existing]	User Login with JWT ← do FIRST	5pt	HIGH
US-E002	[Existing]	Create User (Admin) (security)	3pt	HIGH
US-E003	[Existing]	List Users + Pagination	3pt	HIGH
US-E005	[Existing]	Deactivate User (30-day) (careful)	5pt	HIGH

TYPE B — Gap Stories

US-G001	[Fix]	3 failing tests in UserServiceTest	2pt	P1
US-G002	[Add]	Input validation on POST /users	3pt	P1
US-G003	[Fix]	N+1 query in searchUsers (UMS-3102)	5pt	P2
US-G006	[Upgrade]	Spring Boot 2.7.x → 3.2.x	13pt	P2

Legacy Code Fixes — Production-Safe Approach

 **Safety Rule:** (1) Write a test capturing CURRENT behavior FIRST → (2) Verify it passes → (3) Make the fix → (4) Verify test still passes. NEVER fix without a test anchor.

Fix Priority Matrix

Fix Type	When	Why	Risk
Failing tests	Sprint 0	Cannot build on broken baseline	LOW
Security gaps (no input validation)	Sprint 0	Customer-facing = injection risk	MED
NPE / crash bugs	Sprint 0	Active reliability risk in production	MED
N+1 queries	Sprint 2	Performance, not currently blocking	MED
Test coverage gaps	Sprint 2	Foundation for future safe changes	LOW
Dead code removal	Sprint 3+	Needs customer use confirmation first	HIGH
Framework upgrades	Dedicated sprint	Full regression required — never bundle	HIGH

Playwright E2E — Three Test Layers

Layer 1: UI Tests

@ui

Browser-based tests against React/Angular UI. Tests real user journeys.

- ▶ Login with each role (ADMIN, MANAGER, CUSTOMER)
- ▶ Admin creates/edits/deactivates users
- ▶ Customer creates and tracks orders
- ▶ Manager updates order status
- ▶ Page Object Model pattern

Layer 2: Internal API Tests

@api

HTTP tests against REST endpoints directly. No browser overhead.

- ▶ Every endpoint, all HTTP verbs
- ▶ Auth flows: valid/invalid/expired tokens
- ▶ All error codes and edge cases
- ▶ Pagination, filtering, sorting
- ▶ Fast — no browser spin-up cost

Layer 3: Customer Contract Tests

@contract

Simulate exactly what CUSTOMER applications do. Most critical layer.

- ▶ Use customer's actual auth method (API Key etc.)
- ▶ Call only contracted endpoints
- ▶ Assert contracted response fields only
- ▶ Additions OK — field removals = breaking
- ▶ **GATES every single deployment**



Customer contract tests **MUST** gate every deployment — a broken contract test = customer's production app broken

Master Checklist — Do In This Order

Week 1 — Pework (No Code Changes)

- 1 Service archaeology scan on each service → SERVICE_PROFILE.md
- 2 Cross-service dependency map → DEPENDENCY_MAP.md
- 3 Test health scan → TEST_HEALTH.md per service
- 4 Write root CLAUDE.md + per-service CLAUDE.md files
- 5 Write ARCHITECTURE.md (as-is, truthful) + DOMAIN_GLOSSARY.md
- 6 Verify Claude answers 4 training test questions correctly

Week 2 — Story Gen + Sprint 0

- 1 Story Agent (legacy mode) on each service → Jira backlog
- 2 Human review: P1 gaps + risk levels
- 3 Sprint 0: ONLY fix P1 gaps (failing tests, security)
- 4 Rerun test health scan — verify baseline improved

Weeks 3-4 — First Autonomous Sprint

- 1 Set up Claude 3.5 Sonnet on each service



Start Tomorrow Morning

Three commands. One morning. Claude will understand your system.

```
# 1. Install Claude Code
npm install -g @anthropic-ai/claude-code && claude login

# 2. Navigate to your most critical production service
cd /path/to/your/most-important-service

# 3. Run the service archaeology scan (read-only)
claude    # then paste the SERVICE PROFILE prompt from Phase 0

# Takes 10-15 min per service. Repeat for all services.
```

3 Weeks

Prework + Sprint 0

3 Gates

Human touchpoints only

100%

Customer contract safety